

Project2 开发文档

- 开发软件的操作系统: Windows 操作系统
- 使用的 IDE: Visual Studio (通过命令行在 cmd 终端运行)

思考题的回答

一、为什么这里用 Player，而不是指向 Player 的指针？

1. 使用 Player 可进行快速的赋值操作，在进行撤销或恢复操作时可以简化赋值代码，使代码更加清晰直观，易于理解；使用指针时每次访问玩家属性还需要解引用，增加了代码的复杂性。
2. Player 生命周期较固定，直接包含它，可以避免因指针操作不当而引发的如悬空指针、内存泄漏等问题，即可以更好地把控内存布局与使用情况，避免内存分配释放、指针有效性维护等问题。
3. 由于双向链表 Op 是用于记录游戏操作状态的，使用 Player 结构体本身可以保证每次记录状态时完整地复制了当时角色的所有相关属性，避免了因指针指向的外部 Player 数据被意外修改（比如在别的地方通过指针修改了角色的体力值等情况）而导致记录的操作状态数据不一致的问题，使得链表中每个节点所记录的游戏状态是独立且完整可靠的。

二、C 语言函数的用处都有哪些？

1. **代码的模块化分工**：C 语言函数能够将程序按照功能划分为不同的模块，诸如 render_the_map 函数负责初始化迷宫布局、move_player 函数负

责移动玩家位置等。每个函数只专注完成一项具体任务，这样使得整个程序的结构更加清晰，不同的功能模块可以独立开发、测试和调试，便于多人协作开发项目，也方便后续对单个功能进行修改和扩展，不会因为一处代码的变动而大面积影响其他不相关的部分。

2. **代码复用**：编写好的函数可以在程序的不同地方被多次调用，避免了重复编写相同功能代码的情况，提高了代码的编写效率，也保证了代码逻辑的一致性。例如，如果有多个地方都需要检查玩家是否碰到障碍物，那么编写一个 `checkCollision` 函数，在各个相关位置调用该函数即可，而不用在每个需要检查碰撞的地方都重新写一遍检查碰撞的逻辑代码。
3. **隐藏实现细节**：函数可以将复杂的实现细节封装起来，对外只暴露输入参数和返回值等必要的接口信息。对于调用函数的代码部分来说，不需要关心函数内部具体是如何实现功能的，只需要按照约定传递正确的参数并获取相应的返回结果就行。这有助于降低程序的复杂度，提高代码的可读性。
4. **便于程序流程控制**：函数调用的顺序和嵌套等方式可以灵活地组织程序的执行流程，通过在合适的地方调用不同的函数，可以按照期望的逻辑顺序来实现整个程序的功能，例如根据不同的游戏状态（如开始、进行中、结束等）调用对应的函数来处理相应的游戏逻辑和显示界面等情况，让程序可以有条不紊地运行。

三、 我们现在在每次响应用户输入时刷新 UI。假设游戏里有一个会自己周期性移动的 NPC，那怎么刷新界面效果才会好呢？

可以采用定时刷新机制与事件驱动相结合来刷新界，具体如下：

1. **定时刷新机制**：可以利用定时器（例如在一些支持多线程或者定时中断的编程环境下），设置一个固定的时间间隔（比如每隔一定的毫秒数）触发一次界面刷新操作。在定时器的回调函数或者定时任务处理函数中，去检查游戏中所有需要动态更新的元素状态，包括这个会周期性移动的 NPC 的位置等信息，然后根据新的状态来更新 UI 界面显示。这样能保证 UI 界面按照一定的节奏保持更新，即使没有用户输入操作，NPC 的移动等动态变化也能及时展示在界面上。

同时注意设置合理的时间间隔，间隔太短可能会消耗过多的系统资源，导致程序性能下降；间隔太长则可能会让 NPC 的移动看起来不够流畅，出现明显的卡顿感，影响游戏的视觉体验。

2. **事件驱动结合**：除了定时刷新，依然保留基于用户输入来刷新 UI 的机制。同时，当游戏中的 NPC 发生移动等关键状态变化时，触发对应的事件通知机制，让 UI 模块接收到这个事件信号后，立即执行刷新操作。例如，在 NPC 移动的代码逻辑中，在完成位置更新等关键操作后，会触发相关逻辑，使 UI 模块接收到相关信号，一旦接收到就会及时更新界面来展示 NPC 新的位置状态。

这种事件驱动的方式可以保证在 NPC 状态发生重要变化的第一时间更新 UI，避免了仅仅依靠定时刷新可能出现的更新不及时的情况（比如刚好在定时器两次触发间隔内 NPC 移动了，但界面没能及时更新），结合定时刷新，可以更灵活且高效地保证界面始终能准确反映游戏内包括 NPC 移动在内的各种动态情况。

四、结构化 UI 设计一定好吗？会不会在某一些情况下，我们被迫要作出一些非结构化设计呢？

结构化 UI 设计通常有诸多好处，如代码的可维护性强、功能模块清晰、便于扩展和调试等，像此次 Pj 将 UI 和游戏逻辑分离，通过各种函数来封装不同的 UI 操作，使得游戏逻辑代码相对简洁，不受 UI 频繁变动的影响。

然而，在某些特定情况下可能需要采用非结构化设计更好一些：

1. **性能优化需求极端情况**：当对程序的性能要求达到非常苛刻的程度时，结构化设计中一些函数调用、数据传递等带来的额外开销（比如函数调用栈的操作、参数传递的内存复制等）可能成为性能瓶颈。为了极致地提升性能，可能不得不打破结构化的规范，将一些原本分离的 UI 逻辑和游戏逻辑代码进行合并或者采用更直接、紧凑但相对缺乏结构的代码写法，减少不必要的中间环节开销来提高程序运行速度，例如一些对实时性要求极高的游戏场景下，每一毫秒的性能提升都很关键。
2. **快速原型开发或简单功能实现**：在项目初期进行快速原型开发，只是为了简单验证某个 UI 功能或者游戏玩法是否可行时，过于拘泥于结构化设计会花费较多时间在代码架构搭建上，而采用简单直接的非结构化设计可以更快地实现功能，快速看到效果，后续再根据实际情况决定是否重构为更结构化的设计。
3. **特殊硬件或平台限制**：如果开发的程序要运行在一些特殊的硬件平台上，这些平台可能对代码的内存占用、指令格式等有特殊限制，导致无法很好地实现标准的结构化 UI 设计。例如，某些嵌入式设备内存非常有限，按

照常规的结构化方式去分配函数、结构体等可能会超出内存容量，此时就可能需要根据硬件实际情况，采用更紧凑、非结构化的代码布局来适配硬件资源。

项目背景

“宝藏，嘿嘿，宝藏……” 闹钟声把小黄猛的拉回现实。梦中的金银财宝骤然离他而去，意识到现实的它愤恨地敲击着桌面（以头抢地尔）。也许是用力过猛，消失的秘境居然又重新出现在他眼前。原封不动的场景，甚至被他打翻的宝箱都还在。“这到底是不是梦？”

项目目标

- ✚ 从文件加载地图。
- ✚ 实现撤销和恢复。
- ✚ 能够保存进度。
- ✚ 记录已经完成的关卡。
- ✚ 结构化组织代码。

功能实现

地图加载

在 exe 文件所处文件夹下放置 “level-1 ordinary journey.map”，“level-2 grand journey.map”，“level-3 devil’s journey.map” 三个地图文件，程

序可以读取这三个文件，记录这三个文件的名称形成关卡菜单里的选项，并在用户选择关卡时读取选项对应的地图文件，获取地图信息。

撤销与恢复

可以实现撤销功能，当用户输入 Z 时，小黄会回到上一步的位置，体力值也会回到上一步的值。撤销支持一直撤销直到游戏开始状态。

当用户输入 Y 时，如果有被撤销的下一步。小黄会恢复到下一步的位置，体力值也会恢复到下一步的值。如果用户输入 Z 后又输入了一个合法的移动指令，那么撤销的操作会被清空（如图）。

使用双向链表记录每一步操作的信息，当用户按下 Y, Z 时，使指针 `current` 在这个双向链表的节点上左右移动，把当前状态设置成指针所指的对象内部记录的状态。当发生有效操作时，free 所有的当前指针右侧（next 可达）的节点，让一个描述新状态的新节点成为 next 指针指向的实体，即实现了新操作覆盖旧的被撤销的操作的功能。

进度保存

当用户按 'Q' 或 'q' 需要暂时离开游戏或者用户顺利通关，程序将保留此次游戏记录，并在关卡选择界面做出 (history record) 的提示。当用户选择了保存了进度的关卡，会跳出选择是否加载历史记录菜单，并展示上次游戏结束时的时间与进度信息。如果用户选择是，则恢复到上次退出之前的状态，包括输入模式。如果用户选择否，清空存档，并要求用户选择输入模式。

当程序检测到用户达到退出此轮游戏的条件后，调用 `save_game_progress` 函数，将用户当前信息，包括：时间、地图信息、玩家位置、找到的宝藏数、玩家体力消耗、已选的游戏模式这些信息记录在新建的二进制文件里，并关闭文件。

如果玩家选择恢复历史记录，则调用 `load_game_process` 函数，读取二进制文件里的信息，将其更新至地图结构体与其他相关变量中去。

代码结构与使用说明

1. 该项目由三个地图文件（以及程序运行过程中生成的储存地图信息的二进制文件，定义为 `record_map` 型文件），`main.c` ,`game_logic.c` ,`ui.c`, `file.c` 及其对应的 `.h` 文件组成。
2. 主程序（`main` 函数）主要由大小两个循环，大循环控制用户的游戏设置的选择（处理各个菜单选择对应的逻辑）与游戏回合的跟新（判断是否触发此轮游戏结束的条件），小循环控制用户单次操作的处理与响应（用户处于实时模式或在编程模式下按 `ENTER` 键）。
3. `Game_process`：用于处理游戏相关的核心逻辑；
`File`:用于处理与文件相关的逻辑
4. `Ui` 相关代码：
`welcome_interface`:渲染游戏欢迎界面；
`set_header`：渲染菜单头部；
`render_the_menu`:渲染更新菜单界面

render_the_map:渲染更新地图;

set_hint: 渲染提示语;

set_info_list:渲染信息列表;

set_selection_options:设置菜单具体选项内容;

get_selected_index:获取用户选择选项的索引;

manifest_the_consumption: 显示用户的体力消耗值;

end_the_game:渲染游戏结束界面;